

WarpTrack

WarpTrack is a CUDA-accelerated PyTorch project for distinguishing **muon-like tracks** from **hadronic cosmic-ray shower events** using hit patterns from segmented scintillator detectors.

The long-term goal is to train and validate the model on simulated events, then use the same event representation and inference pipeline on real detector data.

Scope

WarpTrack focuses on one binary classification problem:

```
Muon-like event
      vs
Hadronic shower-like event
```

Each event is represented as a variable collection of detector hits.

A hit may contain:

```
x
y
z
energy deposition
time
module identifier
layer identifier
```

The starter project currently uses:

```
x, y, z, energy
```

The remaining fields are reserved for later integration with real data.

Why this problem?

Muon-like events usually produce a relatively coherent, track-like pattern through multiple detector elements.

Hadronic cosmic-ray showers can produce:

- larger hit multiplicity
- multiple localized clusters
- wider spatial spread
- secondary-particle branches

- less globally track-like geometry

These differences make event topology and local hit relationships useful classification features.

Design philosophy

The project is intentionally split into two parts:

1. PyTorch

- data loading
- model definition
- training
- validation
- inference

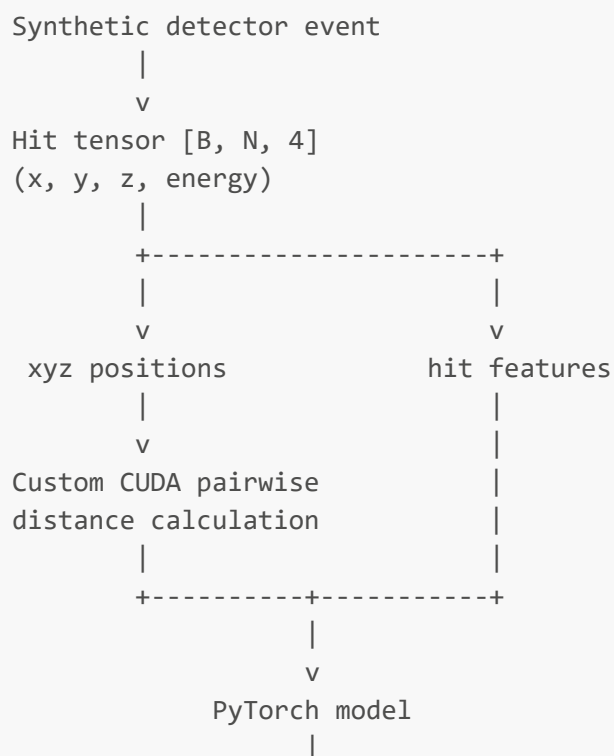
2. Custom CUDA

- geometric operations on detector hits
- pairwise distances
- later radius-neighbor construction
- later graph aggregation

The first CUDA kernel computes pairwise squared distances between hits.

This is not intended to be the final representation. It is the first step toward a CUDA-accelerated graph/point-cloud classifier.

Current pipeline



v
Muon / Hadronic Shower

Repository layout

```
WarpTrack/  
├── cuda/  
│   ├── bindings.cpp  
│   └── pairwise_distance.cu  
├── data/  
│   ├── __init__.py  
│   └── synthetic.py  
├── models/  
│   ├── __init__.py  
│   └── event_classifier.py  
├── warptrack_cuda/  
│   └── __init__.py  
├── benchmarks/  
│   └── benchmark_pairwise.py  
├── tests/  
│   ├── __init__.py  
│   ├── test_dataset.py  
│   └── test_pairwise.py  
├── inspect_event.py  
├── setup.py  
├── train.py  
└── README.md
```

Synthetic data

The starter contains a simplified synthetic event generator.

Muon-like events

Muon-like events are generated from an approximately straight trajectory with:

- small transverse position fluctuations
- moderate event-to-event direction variation
- approximately minimum-ionizing energy deposits
- occasional missing hits

Hadronic shower-like events

Hadronic shower-like events contain:

- a primary shower direction
- increasing spatial spread
- multiple secondary branches
- larger variation in deposited energy

- higher and more variable hit multiplicity

These are not intended to replace Geant4 or another detector simulation.

The synthetic generator exists so that the CUDA and ML infrastructure can be developed before introducing real simulated or measured detector data.

Variable hit multiplicity

Real detector events do not all contain the same number of hits.

The dataset therefore returns:

```
hits : [max_hits, 4]
mask : [max_hits]
label: scalar
```

Unused rows are zero padded.

The mask identifies which hits are real.

This makes the data-loader interface compatible with future real events.

Build requirements

WarpTrack uses a native PyTorch C++/CUDA extension. On Windows, the build requires:

- Python 3.10+
- NVIDIA GPU
- CUDA Toolkit
- CUDA-enabled PyTorch
- Visual Studio 2022 Build Tools with the C++ toolchain
- a Windows SDK

The current Windows development environment has been verified with CUDA 13.3 and a CUDA-enabled PyTorch build compiled against CUDA 13.0. PyTorch reports this as a minor CUDA-version mismatch and notes that it should normally be compatible.

Verify PyTorch:

```
python -c "import torch; print(torch.__version__);
print(torch.cuda.is_available()); print(torch.version.cuda)"
```

Verify CUDA:

```
where.exe nvcc  
nvcc --version
```

Build the CUDA extension

Windows

`setup.py` automatically initializes the Visual Studio x64 compiler environment when `cl.exe` is not already available. It also sets the PyTorch `DISTUTILS_USE_SDK` requirement internally, so a separate Developer Command Prompt is normally **not required**.

From a normal PowerShell or Command Prompt, first change into the repository root, then build:

```
cd C:\Users\Maddie\Documents\GitHub\WarpTrack  
python setup.py build_ext --inplace
```

`setup.py` automatically runs the full unit-test suite after a successful native-extension build. If any unit test fails, the setup command exits with an error instead of reporting a successful setup.

The order matters: `setup.py` and the `tests` directory live in the WarpTrack repository, so run the build only after changing into that directory.

The Windows build uses the flags required by the current PyTorch/CUDA toolchain:

```
MSVC: /O2 /std:c++20 /Zc:preprocessor  
NVCC: -O3 -std=c++20 -Xcompiler=/Zc:preprocessor
```

These settings are intentional:

- **C++20** is required by the current PyTorch C++ headers.
- `/Zc:preprocessor` enables MSVC's standards-conforming preprocessor, required by CUDA 13.3 CCCL headers.
- `DISTUTILS_USE_SDK=1` prevents PyTorch/setuptools from trying to activate an already configured Visual C++ environment a second time.

If automatic Visual Studio discovery ever fails, the known-good manual fallback is:

```
cmd /k ""C:\Program Files (x86)\Microsoft Visual  
Studio\2022\BuildTools\VC\Auxiliary\Build\vcvars64.bat""
```

Then, inside that Developer Command Prompt:

```
cd /d C:\path\to\WarpTrack
set DISTUTILS_USE_SDK=1
python setup.py build_ext --inplace
```

A clean rebuild can be forced with:

```
Remove-Item -Recurse -Force .\build -ErrorAction SilentlyContinue
python setup.py build_ext --inplace
```

or in `cmd.exe`:

```
rmdir /s /q build
python setup.py build_ext --inplace
```

A successful build creates a Python extension module in the repository root with a name similar to:

```
warptrack_cuda_ext.cp314-win_amd64.pyd
```

Verify that it imports:

```
python -c "import warpttrack_cuda; print('WarpTrack CUDA extension loaded')"
```

Run tests

The tests run automatically at the end of:

```
python setup.py build_ext --inplace
```

You can also rerun them manually without rebuilding the extension:

```
python -m unittest discover tests
```

The CUDA unit test compares WarpTrack's custom pairwise-distance kernel against an equivalent PyTorch calculation. A build is not considered validated until the automatic test step passes.

Inspect synthetic events

```
python inspect_event.py
```

This prints one muon-like and one hadronic shower-like event.

Train

```
python train.py
```

The starter model uses:

- per-hit features
- masked global pooling
- summary statistics from the CUDA distance matrix

This is deliberately more permutation-tolerant than treating the distance matrix as an image.

The final model architecture is expected to evolve into a graph or point-cloud network.

First CUDA kernel

For hits i and j ,

$$d_{ij}^2 = (x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2$$

The CUDA extension receives a tensor with shape:

```
[B, N, 3]
```

and returns:

```
[B, N, N]
```

The mask is handled separately by PyTorch.

Development roadmap

Stage 1 — Infrastructure

- synthetic event generation
- padded variable-length batches
- CUDA/PyTorch extension
- tests
- basic classifier
- benchmark suite

Stage 2 — CUDA optimization

Benchmark and optimize pairwise geometry calculations using:

- different block sizes
- shared-memory tiling
- reduced global-memory traffic
- improved memory layout
- CUDA profiling

Stage 3 — Radius-neighbor kernel

Replace the full $N \times N$ distance matrix with a sparse local-neighborhood representation.

Instead of storing every pair, retain hits satisfying:

$$d_{ij} < r$$

or the k nearest neighbors.

Stage 4 — Point-cloud / graph classifier

Represent each hit as a node with features such as:

```
x
y
z
energy
time
detector metadata
```

Use CUDA-generated neighborhoods for message passing.

Stage 5 — Simulation data

Replace the simplified generator with physically simulated detector events.

Candidate sources include detector Monte Carlo outputs produced with tools such as Geant4.

The ML input API should remain largely unchanged.

Stage 6 — Real detector data

Implement an adapter that converts measured detector events into the same event representation:

```
hits
mask
event metadata
```


The trained classifier can then run inference on real events.

Benchmark goals

We will compare:

```
PyTorch implementation
vs
naive custom CUDA
vs
optimized custom CUDA
```

for increasing event multiplicity.

Questions we want to answer include:

- At what event size does the custom kernel become worthwhile?
- Which CUDA block size performs best?
- How much does shared-memory tiling help?
- Does full pairwise distance calculation become memory limited?
- When should we switch to sparse neighbor construction?

End goal

The final project should look approximately like:

```
Detector event
  |
  v
Hit reconstruction / parsing
  |
  v
Point-cloud representation
  |
  v
CUDA neighbor construction
  |
  v
PyTorch graph / point-cloud model
  |
  v
Muon-like
or
Hadronic shower-like
```

This project is intended to demonstrate practical integration of:

- CUDA C++

- PyTorch
- Python
- GPU profiling
- custom PyTorch extensions
- particle-detector event processing
- machine-learning classification
- simulation-to-real-data model deployment

Run the tests

After the extension builds successfully, run the complete unit-test suite from the repository root:

```
python -m unittest discover tests
```

A normal development cycle is therefore:

```
cd C:\Users\Maddie\Documents\GitHub\WarpTrack
python setup.py build_ext --inplace
python -m unittest discover tests
```

Windows build fallback

If automatic Visual Studio discovery ever fails, initialize the same known-good x64 MSVC environment manually:

```
cmd /k ""C:\Program Files (x86)\Microsoft Visual
Studio\2022\BuildTools\VC\Auxiliary\Build\vcvars64.bat""
```

Then, in the Developer Command Prompt:

```
cd /d C:\Users\Maddie\Documents\GitHub\WarpTrack
set DISTUTILS_USE_SDK=1
python setup.py build_ext --inplace
python -m unittest discover tests
```

Geometry-aware event display

WarpTrack includes a 3D validation display for the current synthetic cosmic-ray geometry.

Install Matplotlib if it is not already available:

```
python -m pip install matplotlib
```

Generate one accepted cosmic-ray event and inspect it:

```
python view_event.py
```

The terminal prints the sampled zenith/azimuth and a chronological hit table with hodoscope, layer, bar, channel, hit midpoint, path length, and time of flight. A Matplotlib 3D window then shows every triangular-prism scintillator, the particle trajectory, and the scintillators reported as hits.

The current rack-U positions and exact detector dimensions are still configurable placeholders. The event display is intended to make geometry and intersection mistakes visually obvious before the synthetic events are used for ML training.

Interactive event display and saved snapshot

`python view_event.py` now creates one event, saves that exact event to `warptrack_event.png`, and then opens the same Matplotlib 3D figure interactively. Drag the 3D view to rotate it and use the Matplotlib toolbar for zoom/pan.

Hit scintillator prisms are highlighted, while the hit midpoint is shown in a separate red marker so the geometric crossing point is easy to distinguish.

The scintillator bars are intended to tessellate without physical gaps. If gaps are visible in the display, that is a geometry/visualization issue rather than an intentional detector feature and should be corrected before using the geometry for synthetic training data.

Tessellated triangular scintillators

Neighboring alternating triangular prisms now use half of the triangle-base width as their center-to-center pitch. This makes adjacent bars share their sloped faces instead of leaving empty wedges. The thin optical wrapping between real bars is intentionally ignored for now.

The same triangle cross-section is used in both perpendicular layers. The default detector depth was adjusted consistently for the 9-bar and 16-bar layers; physical dimensions remain configurable placeholders until measured dimensions are supplied.